Design and Development of Specialized Artificial Neural Networks for Industry-Specific Applications

Chamika Deshan
COMScDS251P-015

Project GitHub URL
https://github.com/deshancha/ann_cw

Google Drive Directory

Trained Models hosted at HuggingFace

Problem Formulation

Domain Application

Here we address the design, optimization and deployment of a lightweight Artificial Neural Network (ANN) based human segmentation on resource constrained mobile hardware. The system serves real world application addressing conference video call and prevent exposing sensitive domestic environment or intellectual property.

ConferenceVideo Call

Applications : Business meetings, telemedicine consultations conducted on mobile devices.
Methodology : ANN Process a live camera video frame and isolate human subject and render natural background blur or subsitute the background with a virtual office background.
Completed demonstration application have been developed here.

Operational Constraints

Unlike cloud based or asynchronous segmentation applications video conferencing domain associates with following real time operational constraints.

Per Frame Synchronization

Inferencing is not a one time operation, it must continuously running for **every single camera frame** at the camera's native capture rate **(**24, 30 FPS**).**

Frame to Frame consistency (Temporal Coherence)

The frame inferencing should maintain decent fps rate. Any latency spike, dropped frame causes immediate jumping segmentation mask. Camera capture and render fps is a constant and mask segmentation not maintaining similar fps would noticeable to human eye if it drops below 20fps.

Hardware

Memory and CPU overheads are resource constraints, especially at the inferencing and rendering components in the pipeline.

Question 1 – Dataset and Exploratory Data Analysis

The project uses the Matting Human Dataset, a human portrait matting benchmark.

Dataset Overview

Data

Paired unstructured image data consisting of RGB human portrait images and ground truth alpha mattes(human contour segmented images based on alpha channel).

Dataset Volume

7700 image-matte pairs ~3.16 GB on disk space.

Data splitting

The dataset partitioning strategy was determined through trial and error approach while considering computational requirements.

Test : 500

500 images in testing allowed equitable benchmarking against the classical GrabCut [4] baseline. Because GrabCut is iterative graphcut algorithm running on the CPU which takes about 200 milliseconds per image. There is a significant performance gap between GrabCut (46.22% IoU) and UNet (98.12% IoU). Therefore 500 test set is statistically sound and keeps the margin of error at 0.15%(calcualation included in repo) under 0.5%.

Training: 6120 (85%)
Validation : 1080 (15%)

In first four models (v1.0 to v1.3) we used standard 70%, 30% split (5040 training images, 2160 validation images). Moving to an 85%, 15% split increased the training data set by 1080 images (+21.4%), providing the network with more data and varying human poses and backgrounds. This change showed a significant improvement in both Mean IOU (97.56%) and Dice Score (98.76%).

Final Ratio - **80% Training, 14% validation, 6% Test**.

Reproducibility & Distribution

The dataset is hosted on Hugging Face under deshanc/unet-mobilenet-portrait-segmentation and can be retrieved via a script

```
sh ./config/hf_get_dataset.sh
```

Self Contained Samples

The full dataset exceed 500MB added sample set with 140 paired images (100 training, 20, validation, 20 test) and packed it into the repository allows immediate training, validation, test, XAI and benchmarking.

EDA and Ground Truth Mask Comparison

To verify that the dataset splits are balanced, a mask comparison were conducted across all 7700 images. The table 1 below contains the result.

Metric	**Training Dataset N = 6120)**	**Validation Dataset (N = 1080)**	**Test Dataset (N = 500)**
Foreground Ratio	51.78%	57.44%	49.49%
Background Ratio	48.22%	42.56%	50.51%
Train vs. Test Foreground Difference	Baseline	N/A	-2.29%
Native Image Resolution	600 x 800 (100%)	600 x 800 (100%)	216 x 288 (100%)
Aspect Ratio	3:4	3:4	3:4

Table 1: Ground Truth Mask & Spatial Distribution Analysis

As depicted in table 1 all 6120 training images human portrait occupy an average of 51.78% of total pixels while background occupy average of 48.22%. 50/50 balance ensures gradient update during neural network training not affected by dominant background or foreground pixels.
Foreground person coverage in test set (49.49%) differs from training(51.78%) by only 2.29%. This shows that test set shares same framing and body scale distributed as the training set.
Validation set has slightly closer portrait framing because it has 57.44% foreground which can effectively handle camera zoom and prevents overfitting to a single body scale.

Errors and Qualitative and Operational consequences

False Positive – Background Falsely classified as Person

Confidential physical backgrounds (family members walking in background, whiteboards with company passwords) are not blurred and goes into the video conference feed.
Severity – Privacy Breach, The system should minimize false positive cases.

False Negative – Person Falsely classified as Background

Part of the user (hands, shoulders, etc…) are erroneously classified as background and blurred out.
Severity – Erosions in the Telepresence, Lower professional credibility.

Metrics and Justifications

The accuracy is not considers as primary matric in CNN usecases due to class imbalance problem.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

For example, if an image has 1M pixels where the human portrait occupy only 1K pixels a model could predict all black background mask which could achieve 99% accuracy.
TP – Predicted Positive and Actual Positive = 0
FP – Predicted Positive but actually Negative = 0
TN – Predicted Negative and Actually Negative = 990000
FN – Predicted Negative but actually Positive = 10000

$$Accuracy = \frac{0+990000}{0+990000+0+10000} = 99\%$$

Here we should use the DICE Score which indicates how well the ground truth and predicted mask overlap, here its 0%. Here it completely ignores True Negative background pixels, completely focuses on object only.
We would use **Intersection Over Union (IOU/Jaccard Index) and DICE Score(F1-Score)** as the primary metrics.

IOU Supporting Factors

A = Ground Truth
B = Predicted Mask

$$IOU\ (A,B) = \frac{|\ A\ \cap\ B\ |}{|\ A\ \cup\ B\ |} = \frac{TP}{TP\ +\ FP\ +\ FN}$$

IOU **penalizes** over segmentation (privacy leak/ by FP ↑) and under segmentation (Erossions/ by TP ↓ and FN ↑).

$$DICE(A,B) = \frac{2\ x\ Overlap}{Actual\ Size\ +\ Predicted\ Size} = \frac{2\ x\ |\ A\ \cap\ B\ |}{|A|\ +\ |B|}$$
$$= \frac{2\ x\ TP}{(TP + FN) + (TP + FP)}$$

According to the DICE formula above it does not contain TN which is the background segment. And also it normalize the overlap agianst the size(Actual Size + Predicted Size), not the size of the image. Hence image is larger or smaller it does not care only the size of the person it matters.

ANN Architecture

Modelling Constraints Considered

1. Multi Level Feature Reuse
   Hair silhouettes and finger boundaries occupy only tiny areas of the pixels soup. High level bottle neck features alone cannot reconstruct the human contour accurately. Model architecture must incorporate higher level spatial feature maps in the decoder stages. This is exactly what comes in the **skip connection in U-NET**.

2. High computational power
   Standard convolution associates with higher parameter count and makes larger models. Larger the model the inferencing time would be larger and directly affecting FPS rate in video conferencing. So we would consider depthwise separable convolution which bring down parameter count drastically and creates small model size. Here we would use **MobileNet which is a** depthwise separable convolution **as the encoding part of the U-Net** for feature extraction.

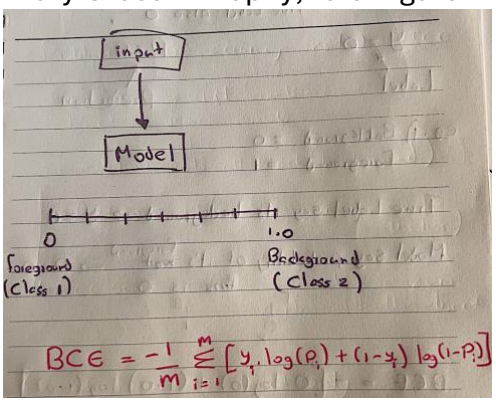
3. Loss Function to address **overlaps and pixel level cross entropy**
   For the overlaps we would use **DICE** score above explained. But pixel level cross entropy is need to tell how bad our model is predicting the mask. This would help the optimizer to figure out how to make the model params to make the model accurate. Since this is binary classifier(Forground/Background) we would use **Binary Cross Entropy**.

$$Loss = Loss\ DICE + Loss\ BCE$$

To address above constraint the chosen architecture is a custom U-NET

1. Encoder- MobilenetV1 as the feature encoder
2. Decoder - Symmetric multi-stage upsampling with Skip connection
3. The loss function would be combined DICE and BCE

Binary Cross Entropy, refer figure 1 below



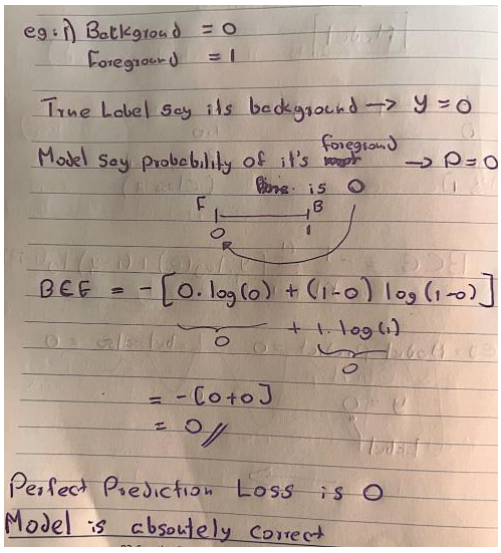


Figure 0: Binary Cross Entrophy and sample usage in binary classifier (Background/Foreground)

MobilenetV1

This is a lightweight architecture with descent accuracy. This was proposed by Google in 2017 [1]. The breakthrough innovation here is the depthwise separable convolution. This distribute the standard convolution operation into two separate steps.

1. Depthwise Convolution
2. Pointwise Convolution

Depthwise Convolution, Figure 1

- Applies single filter to each input channel seperately.
- So it outputs same number of output channels
- Each feature in separate channel learn seperately
- Feature in different channels not getting mixed, depth feature kept intact

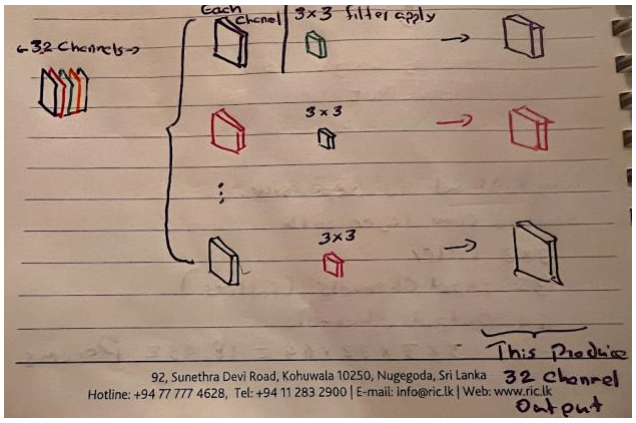
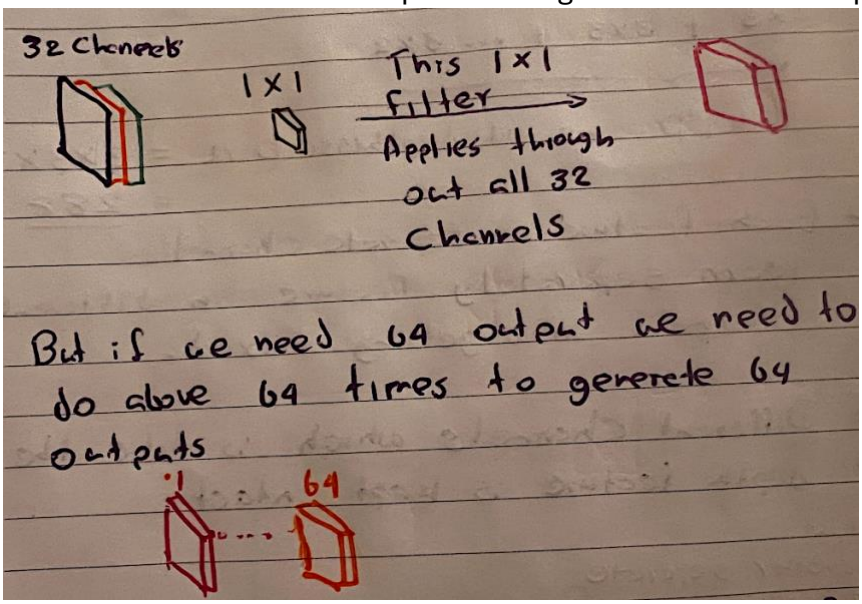


Figure 1: Depthwise Convolution

Pointwise Convolution, Figure 2

- 1 x 1 filter throughout the all channel amd condenses down to single output map
- Different 1 x 1 filters do the same operation to get the number of output channels

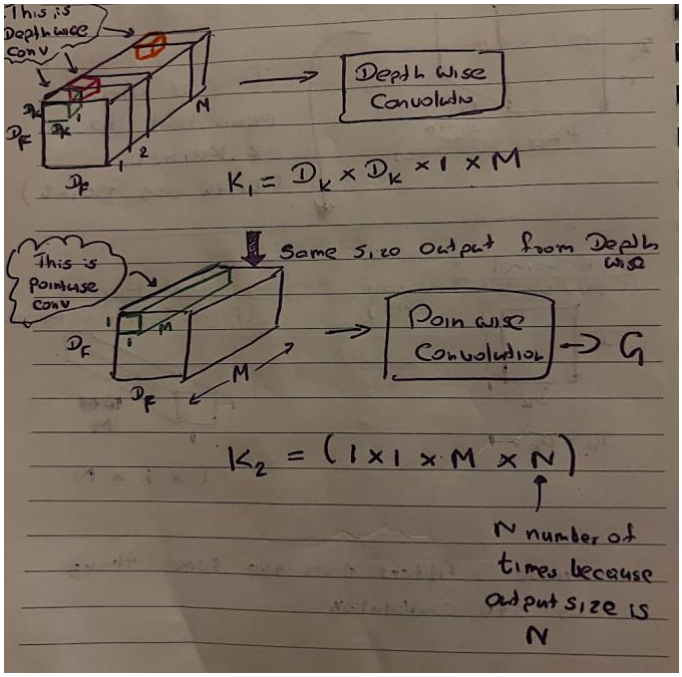


Figurer 2: Pointwise Convolution

The reason the term Seperable

- First it handles spatial filtering independently per channel in depthwise
- Second channel mixing independently per pixel in pointwise

Figure 3 depicts how does the depthwise and pointwise connects each other.



Figurer 3: Depthwise + Pointwise

Cost calculation between Standard Convolution and Depthwise separable convolution

$$COST = \begin{matrix} Number\ of\ Elements\ Per\ Kernel \\ * \\ Number\ of\ Kernels \\ * \\ Number\ of\ Positions\ Taken\ Per\ Kernel \end{matrix}$$

D_f = Input Dimenstion (Width or Hieght)
D_k = Kernel Dimension (Width or Height)
M = Number of Channels
N = Output Size

Cost of Standard Convolution (X_1)

$$X_1 = D_f * D_f * D_k * D_k * M * N$$

Cost of Depthwise Separable Convolution (X_2)

$$X_2 = Depthwise\ Cost + Pointwise\ Cost$$

$$Depthwise\ Cost = D_k * D_k * 1 * M * N * D_f * D_f$$

$$Pointwise\ Cost = 1 * 1 * M * N * D_f * D_f$$

$$X_2 = ({D_k}^2 * M * {D_f}^2) + (M * N * {D_f}^2)$$

$$\text{Cost Ratio} = \frac{X_2}{X_1} = \frac{1}{N} + \frac{1}{{D_k}^2}$$

If we take an example of outputs(N) is 64, Kernels 32, and Kernel dimension (D_k) is 3.
Cost Ratio = 12.67%, Cost Reduction is 87.33%

Param Count Comparison and Model Size

Standard Convolution = 3 x 3 x 32 x 64 = 18432
Depthwise Seperable = (3 x 3 x 32) + (1 x 1 x 32 x 64) = 2336
Depthwise seperable has less params hence smaller size models.

Depth Wise Seperable(DSC) Block

Figure 4 and 5, depicts the DSC block we have in the implementation

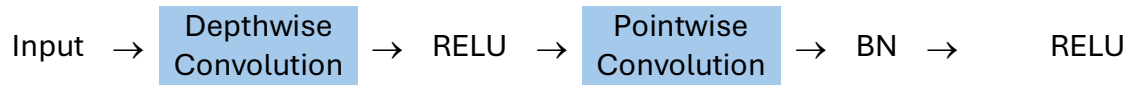


Figure 4: DSC Block

```python
# This is the depthwise separable convolution block, as the paper
# Paper => [F -> StdConv -> Batch Norm -> RELU -> Conv 1x1 -> Batch Norm -> RELU]
def conv_depth_wise_seperable(inp: int, out: int, stride: int) -> nn.Sequential:
    return nn.Sequential(
        # 1. Depthwise: 3x3 spatial filter per input channel
        nn.Conv2d(inp, inp, kernel_size=3, stride=stride, padding=1, groups=inp, bias=False),
        nn.BatchNorm2d(inp),
        nn.ReLU(inplace=True),
        # 2. Pointwise: 1x1 channel-mixing convolution
        nn.Conv2d(inp, out, kernel_size=1, stride=1, padding=0, bias=False),
        nn.BatchNorm2d(out),
        nn.ReLU(inplace=True),
    )
```

Figure 5: DSC Block implementation

MobilenetV1 we used for encoder stage in UNET as decribed above and Table 1 below demonstrate the blocks progression while figure 6 below depicts implementation configuration.

```python
# (in_channels, out_channels, stride)
CONFIG_FOR_13_DEPTHWISE_SEPERABLE_BLOCKS = [
    # Stage 0 (1 block) —> Output: (64, 112, 112)
    (32, 64, 1),
    # Stage 1 (2 blocks) —> Output: (128, 56, 56)
    (64, 128, 2),
    (128, 128, 1),
    # Stage 2 (2 blocks) —> Output: (256, 28, 28)
    (128, 256, 2),
    (256, 256, 1),
    # Stage 3 (6 blocks) —> Output: (512, 14, 14)
    (256, 512, 2),
    (512, 512, 1),
    (512, 512, 1),
    (512, 512, 1),
    (512, 512, 1),
    (512, 512, 1),
    # Stage 4 (2 blocks) —> Output: (1024, 7, 7)
    (512, 1024, 2),
    (1024, 1024, 1),
]
```

Figure 6: DSC block configuration

Stage	Block Index	Configuration (in/out/stride)	Input Tensor	Stride & Size Change	Output Tensor	Feature Map Generated
Inital		conv_standard(3, 32, 2)	(3, 224, 224)	Stride 2 (halves)	(32, 112, 112	
Stage 0	0	(32, 64, 1)	(32, 112, 112)	Stride 1 (Same)	(64, 112, 112)	(64, 112, 112) ***Skip Con***
Stage 1	1	(64, 128, 2)	(64, 112, 112)	Stride 2 (halves)	(128, 56, 56)	
	2	(128, 128, 1)	(128, 56, 56)	Stride 1 (keeps)	(128, 56, 56)	(128, 56, 56) ***Skip Con***
Stage 2	3	(128, 256, 2)	(128, 56, 56)	Stride 2 (halves)	(256, 28, 28)	
	4	(256, 256, 1)	(256, 28, 28)	Stride 1 (Same)	(256, 28, 28)	(256, 28, 28) ***Skip Con***
Stage 3	5	(256, 512, 2)	(256, 28, 28)	Stride 2 (halves)	(512, 14, 14)	
	6	(512, 512, 1)	(512, 14, 14)	Stride 1 (Same)	(512, 14, 14)	
	7	(512, 512, 1)	(512, 14, 14)	Stride 1 (Same)	(512, 14, 14)	
	8	(512, 512, 1)	(512, 14, 14)	Stride 1 (Same)	(512, 14, 14)	
	9	(512, 512, 1)	(512, 14, 14)	Stride 1 (Same)	(512, 14, 14)	
	10	(512, 512, 1)	(512, 14, 14)	Stride 1 (Same)	(512, 14, 14)	(512, 14, 14) ***Skip Con***
Stage 4	11	(512, 1025, 2)	1024	Stride 2 (halves)	(1024, 7, 7)	
	12	(10241 1024, 1)	1024	Stride 1 (Same)	(1024, 7, 7)	(1024, 7, 7) Bottleneck

Table 2 : Blocks Progression

Every encoder generated feature map marked as a skip connection in Table 1 concatenated with the corresponding U-Net decoder upsampled feature map. The main reason for this is recover dropped spatial representations during downsampling, such as hair silhouettes, finger boundaries, etc...
We do not have pooling here, polling is for downsampling the spatial dimesntion. When we go deep in the network number of channels increase and spatial dimensions decrease as depicted in Table 2 above. To decrease spatial dimension we have used stride = 2 and depthwise convolution instead polling.

Shrinking Hyperparameters

To make the model even faster/smaller there are two hyperparameters discussed in Mobilenetv1 paper[1].

1. Width Multiplier - α

2. Resolution Multiplier - ρ

Width Multiplier α

This reduces the number of Kernels, hence reduce the depth of the output and model size as depicted in figure 7 below.

$\alpha \in [0, 1]$

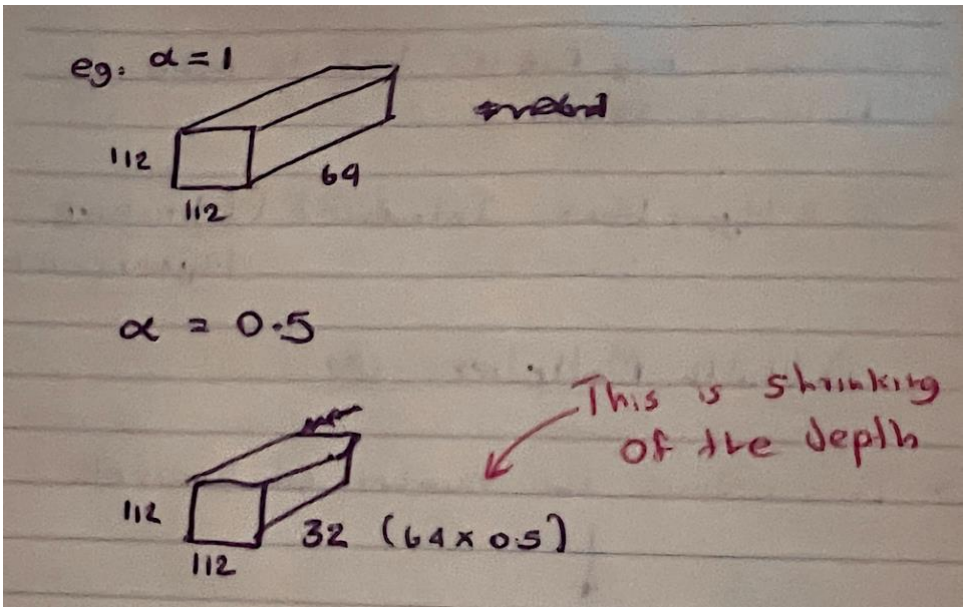


Figure 7: Width Multiplier

When $\alpha = 1$, No Shrinking. We refer to this as Baseline model in MobilenetV1. During out model developmenet also we used α with 0. 5 and 0.75.
Cost Computation with Width Multiplier
Depthwise Separable Convolution Cost Computation, X

$$X = ({D_k}^2 * M * {D_f}^2) + (M * N * {D_f}^2)$$

When Consider Alpha We need to consider parameters representing number of kernels

1. M = Depthwise Kernels
2. N = Pointwise Kernels

$$X_\alpha = ({D_k}^2 * \boldsymbol{\alpha}M * {D_f}^2) + (\boldsymbol{\alpha}M * \boldsymbol{\alpha}N * {D_f}^2)$$

Here the reduction is approximately equals to $\boldsymbol{\alpha^2}$, the calculation depicted in figure 8 below.

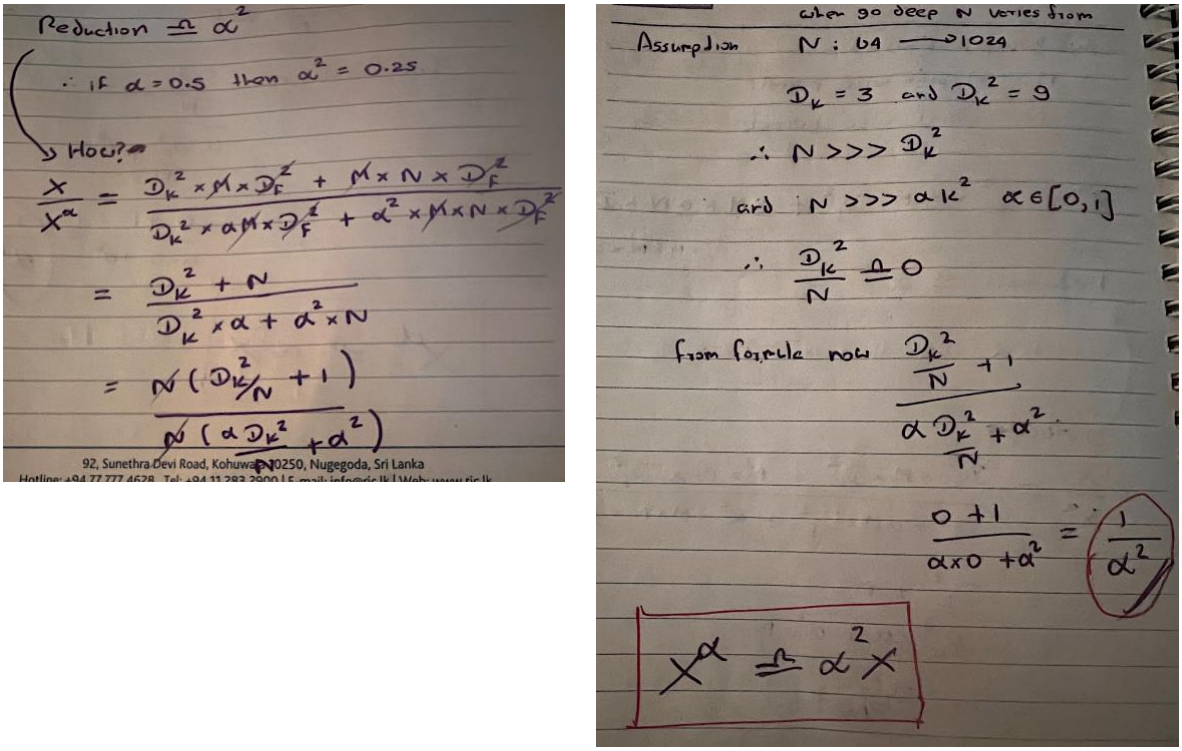


Figure 8: Width multiplier Cost Redcution calculation

Resolution Multiplier ρ

The resolution we refer here is the input dimension (D_f). So we multiply all D_f by ρ. And as similar to width multiplier cost is reduced by ρ^2. Resolution multiplier makes the feature maps spatially smaller.
MobilenetV1 paper also discussed about reducing the number of layers and make shallow models. But in comparison thinner models better than shallow models by 3%[1]. So we only used width multiplier when creating out model.

U-Net Architecture [2]

U-Net is a symmetrical “U” shape encoder-decoder convolution neural network designed for pixel level predictions. We used U-Net here for segmentation of foreground and background from human portrait images.
U-Net Components

1. Encoder
2. Bottleneck
3. Decoder/Upsampling
4. Skip Connection

Encoder

As discussed above the encoder part which is for efeature extraction we used mobilenet.

Bottleneck

The bottom part of the U-Net connecting encoder and the decoder. It has the most compressed feature set with highest channel count.

Decoder/Upsampling

This steps recovers the spatial resolution which was downgraded by encoding above. There are few upsampling methods

1. Nearest Neighbour
2. Transpose convolution
   Unlike nearest neighbour this has another 5 trainable weights to fine tune during the training as depicted in figure 9 below.

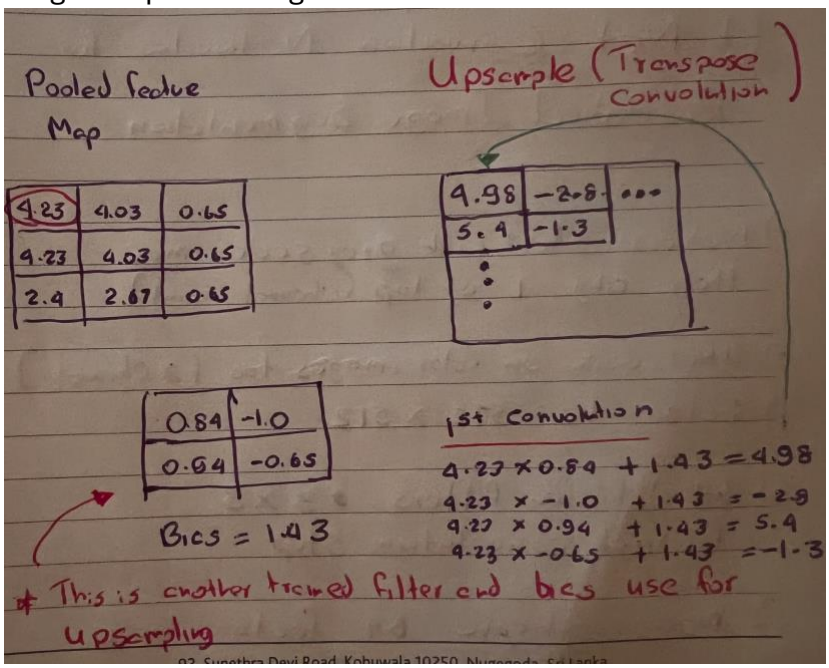


Figure 9: Transpose convolution upsampling

3. Bilenear Interpolation
   This is the linear interpolation(weight based) considering both x and y axises. In our model we would use this instead transpose convolution due to associated grid-like or checkerboard artifacts[3].

SKIP Connection

The upsampled feature map is concatenated along the U axis with the corresponding feature map extracted from the encoder. Concatenating the encoder feature maps we highlighted in the table 2 above. First it we need to do a final convolution with a kernel (1 x 1 x 2) and concat as depicted in figure 10 below.

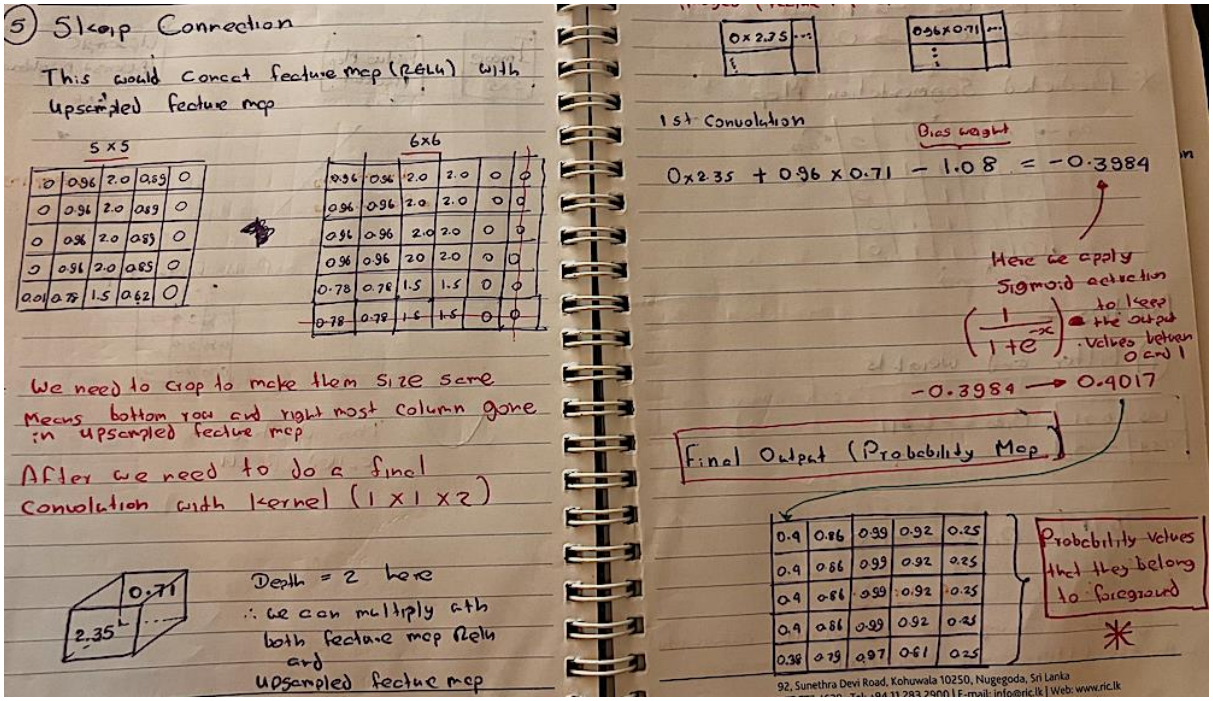


Figure 10: U-Net Skip connection

Alternative Approaches

Table 1.3 below shows possible alternative approaches with different CNN/Classical architectures and the reson why they were rejected on our conference call usecase.

Architecture	Parameters	Rejection
Classical GrabCut	None	Incompatible for video 30fps
DeepLabv3-ResNet	58.6M	Unnecessarily heavy
DeepLabv3-MobileNetV3	11M	Lack of skip connection to fine tune hair edges, fingers, etc..
MobileNetV1 + Unet (Our approach)	8.3M (W = 1.0) 3.9M (W=0.5)	FPS still low without backing up from apple neural engine when inferencing
MobileNetV3 + Unet (Our approach)	0.98M	Selected due to higher fps without apple neural engine backing up

Table 3: Alternative CNN/Classical approaches

Following benchmarking MobilenetV1 against other architectures highlighted in MobilenetV1 Paper[1]

1. Baseline MobileNet (W = 1.0) is **32x smaller** and requires **27x less computation** than VGG-16 while achieving +1.0% higher accuracy
2. Baseline MobileNet is more accurate than GoogLeNet while consuming **2.5x less computations**
3. Scaled thinner MobileNets (W = 0.5) outperform SqueezeNet and AlexNet in both speed and accuracy

Model Development

Evolutionary Development Milestones

1. Milestone 1– **v1.0**
   MobileNetV1 (W = 1.0) Trained from scratch

- 5040 training and 2160 validation images
- Adam Optimizer
- BCE + DICE Loss
- Learning Rate 0.001
- 3 Epochs

2. Milestone 2 – **v1.1 (Baseline)**
   Same like v1.0 but 15 Epochs
3. Milestone 3 – **v1.2**
   Transfer Learning with pretrained MobileNetV3 backbone integrated into the U-Net decoder with same like baseline but 15 Epochs
4. Milestone 4 – **v1.3**
   V1.1 with width multiplier (α =0.5) with the intention of thinning the model.
5. Milestone 5 – **v1.4**
   MobilenetV3 backbone in v1.3 above retrained with new training/validation imageset partition(85:15 - 6120 training, 1080 validation)

Model Creation Execution

As shown in the configuration files in figure11.a below the .env.gloabl needs to update with image data set path and target model version configuration depicted in figure 11.b below. Here it's model v1.4 above.

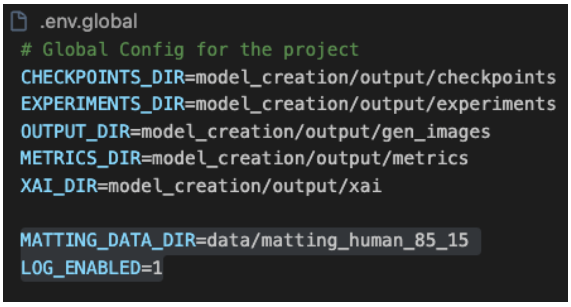


Figure 11.a : Global Config

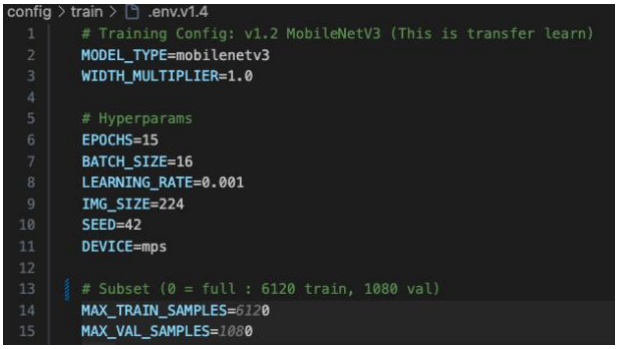


Figure 11.b : Model v1.4 configuration

Once configured below command could run on the root directory to create the model. For each model a separate configuration file is added under config/train directory.

```
python model_creation/main.py --env config/train/.env.v1.4 --task train
```

Model Deployment and Versioning

For model versioning initially started with DVC and later move to hugging face. All models deployed under public hugging face model repo.
Execute below to get all trained models from hugging face to respective directories.

```
sh ./config/hf_get_all.sh
```

Model Evaluation against 500 test image set

Same as training a configuration files are added for evaluation of each model created as shown in figure 12 below.

config > test > .env.v1.4

```
# Test Config: v1.2 MobileNetV3 Large
MODEL_TYPE=mobilenetv3
WIDTH_MULTIPLIER=1.0
CHECKPOINT_PATH=model_creation/output/all_trained/v1.4_mobilenetv3_best.pt

IMG_SIZE=224
BATCH_SIZE=1
MAX_TEST_SAMPLES=500
DEVICE=mps
```

Figure 12 : Model Testing configuration (v.1.4)

The model evaluation result depicted in table 4 below compares Mean IOU and DICE Score of each model when tested against 500 images. The original, ground truth and predicted mask also included on best and worst cases with corresponding IOU/DICE values. The reason why the model picked different worst and best image for each time it ran is they has different architectures, hyper params which fundamentally changes where they fail/success.

Model	Mean IOU	Mean DICE
V1.0	87.59%	93.14%

Original	Best/Worst Sample Ground Truth	Predicted
		
		IOU: 0.9843 DICE: 0.9921
		
		IOU: 0.5433 DICE: 0.7040

V1.1 96.78% 98.33%





IOU: 0.9917
DICE: 0.9958





IOU: 0.6118
DICE: 0.7591

V1.2 97.38% 98.66%





IOU: 0.9916
DICE: 0.9957





IOU: 0.8501
DICE: 0.9189

V1.3 95.51% 97.65%





IOU: 0. 9907
DICE: 0.9953

V1.4 97.56% 98.76%

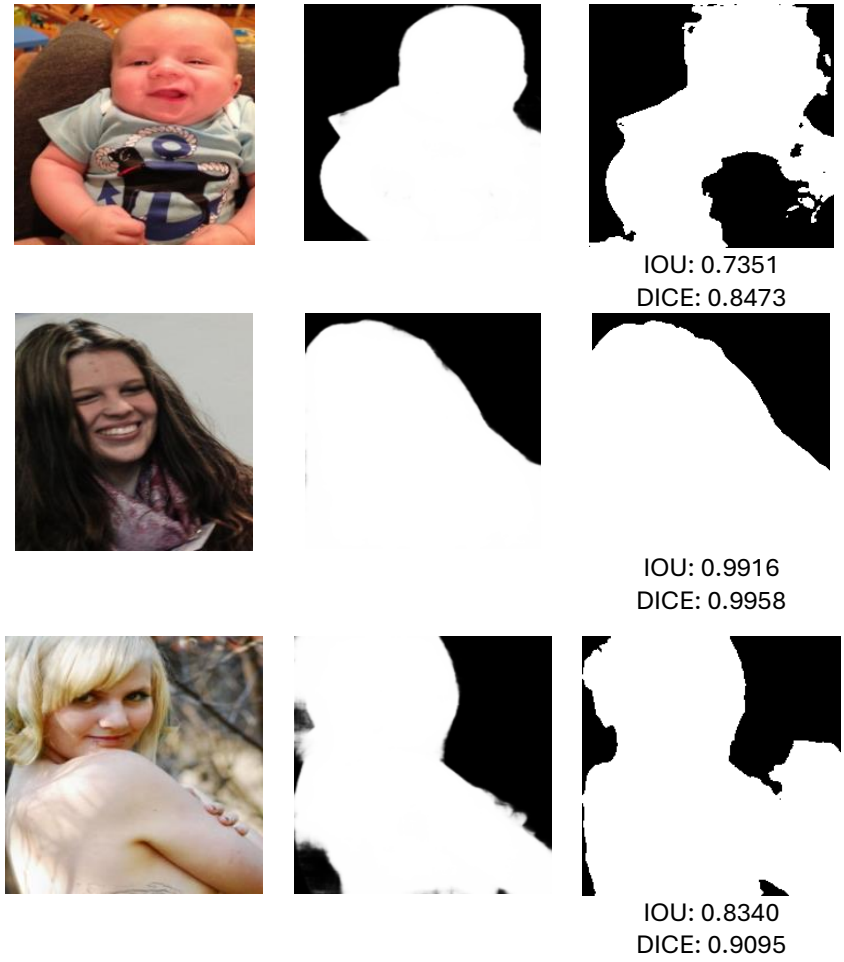


Table 4: Trained 5 models IOU/DICE comparison and Best/Worst IOU images

	Model Family	Train/Validation Split	Width Multiplier (α)	Epochs	Params	Mean IOU	Mean DICE
V1.0	MobileNetV1	70:30	1.0	3	8.31M	87.59 %	93.14 %
V1.1	MobileNetV1	70:30	1.0	15	8.31M	96.78 %	98.33 %
V1.2	MobileNetV3	70:30	1.0	15	0.98M	97.38 %	98.66 %
V1.3	MobileNetV1	70:30	0.5	15	3.96M	95.51 %	97.65 %
V1.4	MobileNetV3	85:15	1.0	15	0.98M	97.56 %	98.76 %

Table 5: Model comparative evaluation

Quantitave Analysis based on Table 5 model comparative evaluation

1. Pretrained MobilenetV4 (v1.4) has the highest predictive accuracy with 97.56% Mean IOU and 98.76% DICE Score. Its +0.78% IOU over MobilenetV1 baseline(v1.1) and 8x fewer parameters count, which **v1.4** would give very good performance in our video conferencing usecase.

2. α = 0.5 (v1.3) reduced the paramer count by 52% when compare with α = 1.0 (v1.1), but it has reduced the IOU/DICE value signifiantly when compare with MobilenetV3(v1.4 and v1.2).

GrabCut[4] Benchmark

Here we have benchmarked the v1.4 model against Grabcut classical baseline on 500 test data set. And table 6 below depicts the results.

Method	Mean IOU	Mean DICE
GrabCut	46.22%	61.3%
Model V1.4	98.14%	99.05%

Table 6: Grabcut vs CNN model v1.4

Version 1.4 outperforms the Grabcut by +51.92% IOU. Its noticeable that Mean IOU/DICE of model v1.4 on table 5 and table 6 are different on same 500 test image data set. The reason is on benchmark result which is on table 6 above IOU/DICE compare against binarized groud truth image. Hence Grabcut only outputs binary mask, model generated mask also converter to binary with a threshold. At boundy pixels where 0.7 like fraction alpha becomes full 1.0 value. This is called **Soft IOU and Binary IOU**.

Figure 13 below illustrates a Grabcut and CNN Model v1.4 visual comparison.

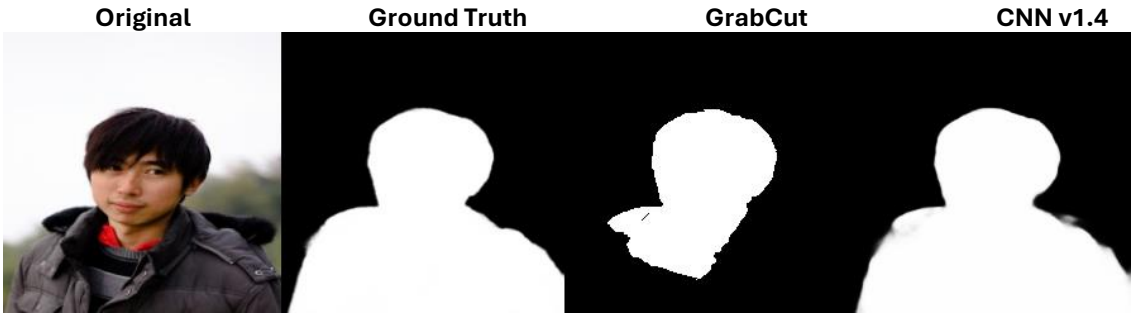


Figure 13: Grabcut vs CNN v1.4

This can be executed from the root directory by running below command

```
python model_creation/main.py --env config/benchmark/.env.v1.4 --task benchmark
```

Application Implementation

Production grade Native iOS application written with ObjectiveC/C++ with hardware accelerated inferencing and processing. The architecture is illustrated in figure 14 below.

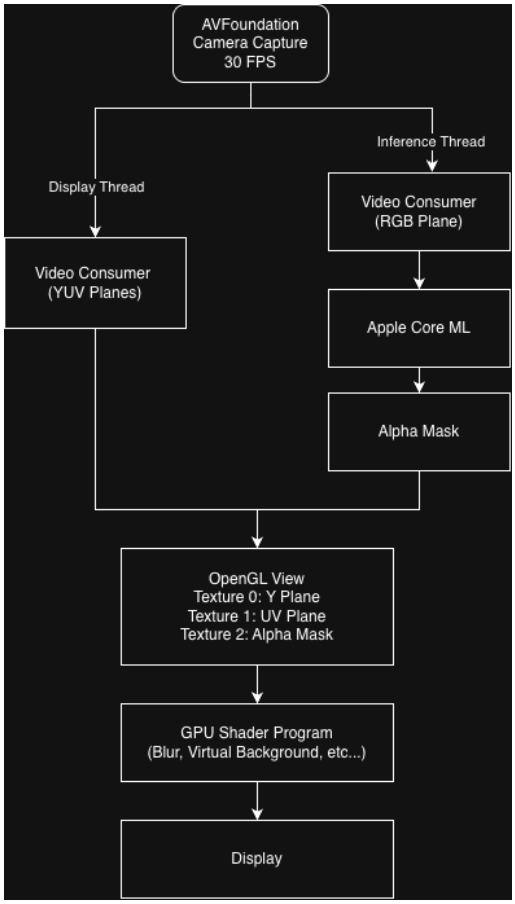


Figure 14: App architecture

1. Local/Network Video Pipelines
   RTP/UDP based video pipeline with fully network streaming compatible. Local camera frame or network streaming video supported.
2. Asynchronous Inferencing
   Frame capturing and inferencing run on completely independent threads synchronized via thread safe mutext. The intention was to individually capture framerates of rendering and inferencing for benchmarking purposes.
3. Hardware accelerated Model
   Pytorch model exported to Apple Core ML enabling execution on the dedicated **Apple Neural Engine**
4. GPU Processing
   After generating the mask the alpha pixel isolated and processed via a fragment shader program which runs on GPU. Rendeing is not a burden to CPU

CPU usage Profiling

iPhone 13 with Apple Neural Engine (Figure 15)

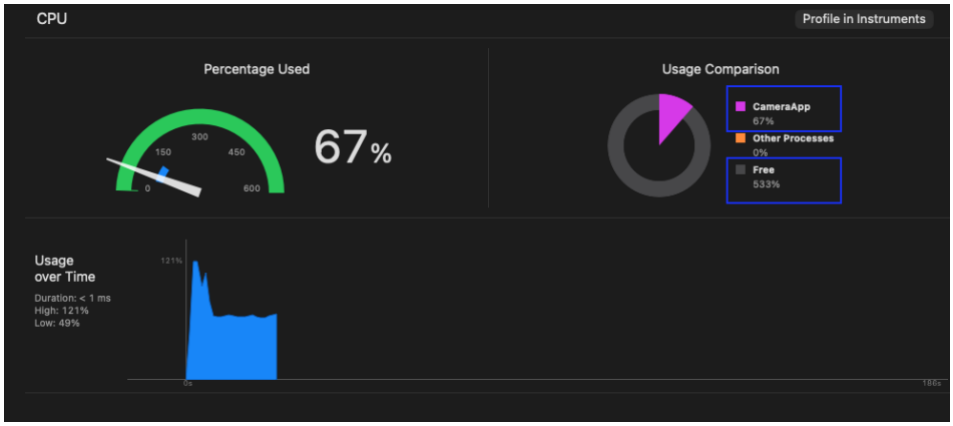


Figure 15: CPU Profiling on iPhone 13

Apple A15 Chip has 6 Core CPU = total capacity 600%
App using 67%, while 533% is free.

$$Total\ CPU\ Load = \frac{67\%}{600\%} \simeq 11\%$$

iPad Pro 10.5 No neural Engine (Figure 16)

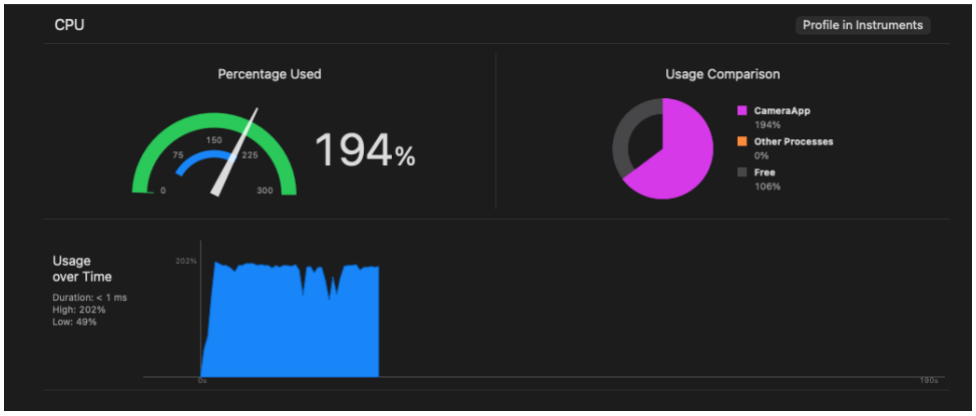


Figure 16: CPU Profiling on iPad Pro 10.5

$$Total\ CPU\ Load = \frac{194\%}{400\%} \simeq 48\%$$

Device	Chip	Cores	Neural Engine	Processing GPU	Camera FPS	CPU Load	FPS	
							Render	Inference
iPhone 13	A15	6	Yes	Yes	60	**10%**	**60**	**60**
iPad Pro 10.5	A10X	4	No	Yes	60	**48%**	**60**	**40**

Table 7: Performance Matrix

Quantitative analysis based on performance matrix in table 7

1. Neural engine backed hardware highly affects the inference performance which has almost no delay in iPhone 13.

2. Even without neural engine backing still the light weight model(v1.4) able to maintain 40 fps on > 10 years old hardware.

Explainable AI (XAI)

Unlike Linear Regression which is self explanatory Neural networks are less self explanatory due to they have millions of parameters. In our model also it has 0.98 Million parameters. Therefore even it has 97.56% IOU we have no idea why it has succeeded.
Questions in our human segmentation model

1. Did it actually learn human features
2. Was it cheating by using a shortcut

Here we can use XAI to prove that the model is making decisions based on actural human features that it learnt, not by coindidences.

IOU = How Well Model Performed
XAI = Why it Performed well or why failed

We picked Grad-Cam[5] gradent ased localization method for XAI and following explans the reason

1. **Latency**
   SHAP/LIME uses super pixel method to minimize the number of combinations. Our input image size 224 x 224 (50176), 2^{50176} combination is impossible calculation and lets assume it segment the image into 50 super pixel segment. Still 2^{50} is a huge number for calculation.
   Even it uses Kernel Level Shap still it could associate with many combinations and calculations.
   Grad-Cam uses only one backward pass and one forward pass which is super fast and less calculation.

2. **When LIME/SHAP** create super pixels it destroys fine boundaries. Therefore Hair/Shoulders pixel level precision gone.

3. **Grad-Cam offers architectural insights**
   It can goes to the mobilenet encoder stages and explain what convolution feature map actually learned.

In our model Grad-Cam would be use to explain what part of the image used to make the segmentation by highlighting pixel regions of the image that important for segmentation by drawing HeatMap(Class Activation Map/Cams).

Grad-Cam Architecture

Here it generates the CAMS using weighting feature maps in convolution layer by class gradient as depicted in Figure 17 below.
Below is the process of calcualting the weight (a_k^c) of the feature map

- Convolution Layer has 64 Feature Maps
- A^k = Arbitrary Map from 64 maps
- Grad-Cam is based on gradient of the predicted(Logit) for Class C
- I,j are arbitrary position in the feature map matrix (A^k)
- Z = Total elements in feature map
- $A_{i,j}^k$ = Value for arbitrary position in the feature map
- With back propagation we can find the derivative of logit (Y^c) with respective to the element of the map

$$\frac{d\,y^c}{d\,A_{ij}^k}$$

- This gradient tells us how much logit changes with small changes in the element
- **Larger gradient** indicate that particular element **important for the logit**
- Taking the **average of all gradient** we can undetand how important entire feature map is. This is our **weight** $(\boldsymbol{a_k^c})\ of\ the\ feature\ map.$

$$\boldsymbol{a_k^c} = \frac{\sum_i \sum_j \frac{d\,y^c}{d\,A_{ij}^k}}{\boldsymbol{z}}$$

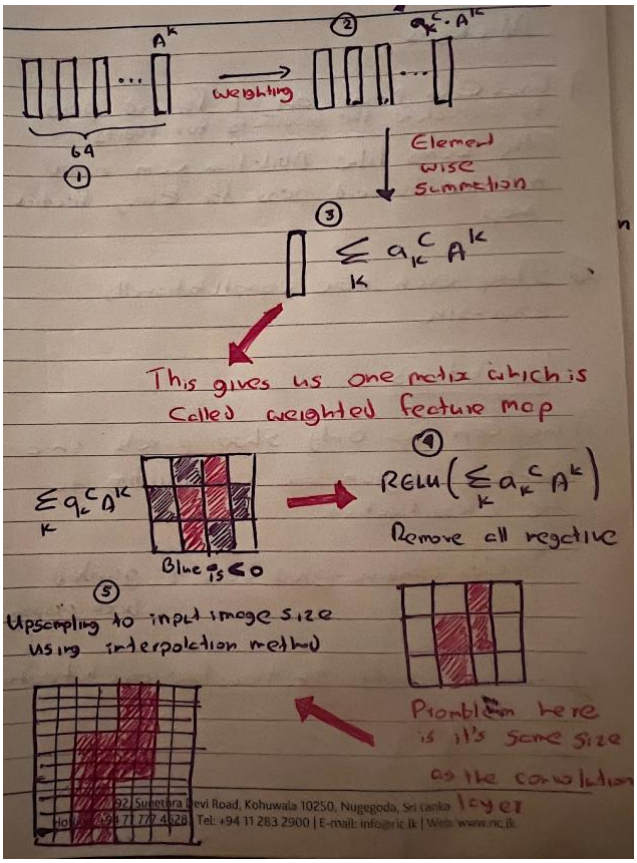


Figure 17: Grad-Cam Heat Map Generation

Execute Grad-CAM

Here we have created .env config for execute Grad-CAM for each model we creatred, Figure 18 shows the v1.4 model.

config > xai > .env.v1.4

```
# Explainable AI (XAI) Config: v1.4 MobileNetV3 (Width=1.0, Epoch = 15)
MODEL_TYPE=mobilenetv3
WIDTH_MULTIPLIER=1.0
CHECKPOINT_PATH=model_creation/output/all_trained/v1.4_mobilenetv3_best.pt
IMG_SIZE=224
DEVICE=mps

# Best and Worst images identified during test
# Make sure to run : python model_creation/main.py --env config/test/.env.v1.4 --task test
BEST_ORIG_PATH=model_creation/output/metrics/test_best_orig.png
BEST_GT_PATH=model_creation/output/metrics/test_best_gt.png

WORST_ORIG_PATH=model_creation/output/metrics/test_worst_orig.png
WORST_GT_PATH=model_creation/output/metrics/test_worst_gt.png
```

Figure 18: Configuration for XAI execution for model v1.4

Grad-CAM Run Steps

1. Isolate the maximum scoring prediction(image) and minimum scoring prediction image) from 500 test image set for each model.

```
python model_creation/main.py --env config/test/.env.v1.4 --task test
```

2. XAI examine the failure mode unique to each model.

```
python model_creation/main.py --env config/test/.env.v1.4 --task xai
```

XAI Comparative Table and Failure Case Analysis

Version	Architecture	Best IOU	Best DICE	Worst IOU	Worst DICE	Worst Case Delta IoU
1.0	MobilenetV1 (3 Epoch)	98.73%	99.36%	54.35%	70.42%	Baseline
1.1	MobilenetV1 (15 Ep, $\alpha = 1.0$)	99.75%	99.87%	61.01%	75.78%	+6.66% over v1.0
1.2	MobilenetV3 (Transfer Learn, $\alpha = 1.0$)	99.54%	99.77%	85.33%	92.08%	+24.32% over v1.1
1.3	MobilenetV1 (15 Ep, $\alpha = 0.5$	99.58%	99.79%	73.34%	84.62%	-11.99% below v1.2
1.4	MobilenetV3 (85/15 data set)	99.48%	99.79%	83.57%	91.05%	

Table 8: XAI Importance comparison

Best Case vs Worst Case

All models scored over 98.73% on best case IOU, this confirm the true test of neural network architectres is the testing of worst-case(how gracefully it fails).

Transfer Learning

Transfer learning of MobilenetV3 (v1.2) is superior to training from scratch in MobilenetV1(v1.1) . Worst Case IOU rise from 61.01% to 85.33%.

Table 9 below illustrates the Grad-CAM generated heat map for each model best/worst cases

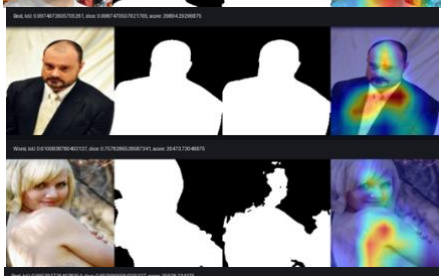
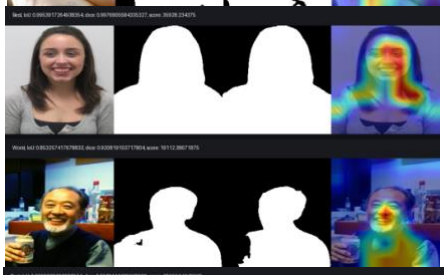
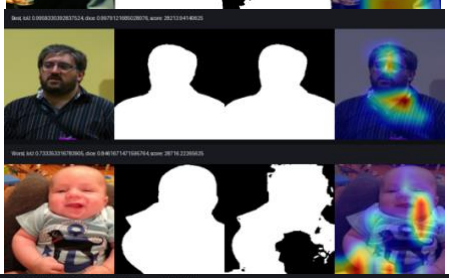
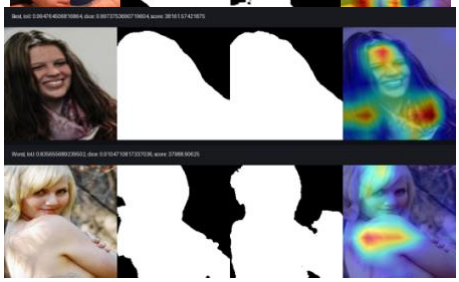
Model	Heat Map
V1.0	
V1.1	
V1.2	
V1.3	
V1.4	

Table 9: Grad-CAM generated heat map for each model best/worst cases

MobileNetV3-UNet vs Pretrained Vision Foundation Model

We evaluate MobileSAM[6] as a Pretrained Vision Foundation Model . This is derived from Meta AI's Segment Anything Model (SAM). The model pretrained on the massive dataset (11M images, 1.1B masks). MobileSAM compressed original SAM's vision transformer into a small model with 9.66M parameters.

MobileNetV3-UNET vs MobileSAM (500 Test Iamges on Apple Sillicon MPS)

Table 10 below illustrates the 500 test data benchmark on our trained U-Net-MobileNet3 v1.14 above against MobileSam and in all dimensions the winner is our model U-Net 1.4

	UNET (v1.4)	**MobileSAM**	**Diff/Ratio**	**Winner**
Param	0.98 M	10.13 M	10x Smaller	v1.4
Mean IoU	98.14%	73.34%	+24.80%	v1.4
Mean Dice	99.05%	76.87%	+22.18%	v1.4
Min IOU	83.58%	0.00%	+83.58%	v1.4
Max IOU	99.65%	99.53%	+0.12%	v1.4
FPS	100	5.7	17X Faster	v1.4

- The main reason is MobileSam was trained as an "Segment-Anything" model. It segements sub parts like Face, Shirt, etc.. rather than complete human contour. But U-Net learned semantic boundaries from maint images directly.
- From all 500 images U-Net never suffered 0% IOU, but MobileSam does.
- For real time prediction like video conference MobileSam not a good option when compare with Unet whcn we consider the latency. U-Net easily achived 100 fps on Mac mps when Mobile Sam only 5 fps.

Figure 19 below shows the above found best/worst images of U-Net v1.4 images against MobileSam. MobileSam gives even worse IOU values.



Figure 19: Unet Best/Worst Images against MobileSam

References

[1] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv preprint arXiv:1704.04861, 2017.
[2] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, Munich, Germany, 2015, pp. 234–241. doi: 10.1007/978-3-319-24574-4_28.
[3] A. Odena, V. Dumoulin, and C. Olah, “Deconvolution and checkerboard artifacts,” Distill, vol. 1, no. 10, Art. no. e3, 2016, doi: 10.23915/distill.00003.
[4] C. Rother, V. Kolmogorov, and A. Blake, "GrabCut: Interactive foreground extraction using iterated graph cuts," ACM Transactions on Graphics (TOG), vol. 23, no. 3, pp. 309–314, Aug. 2004
[5] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), Venice, Italy, 2017, pp. 618–626, doi: 10.1109/ICCV.2017.74.
[6] C. Zhang, D. Han, Y. Qiao, J. U. Kim, S.-H. Bae, S. Lee, and C. S. Hong, "Faster segment anything: Towards lightweight SAM for mobile applications," arXiv preprint arXiv:2306.14289, 2023. doi: 10.48550/arXiv.2306.14289.